МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

РОССИЙСКОЙ ФЕДЕРАЦИИ

СТАРООСКОЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ

ИНСТИТУТ ИМ. А.А. УГАРОВА

(филиал) федерального государственного автономного образовательного учреждения высшего образования «Национальный исследовательский технологический университет «МИСИС»

ОСКОЛЬСКИЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ

Отчет

по индивидуальному проекту
на тему: «Разработка веб-проекта для контроля посещаемости и успеваемости студента»

Выполнил:

студент группы ИСП 25-4

Ковалев М.В

Проверил:

преподаватель Артюхина Д. Д.

Старый Оскол, 2026

Введение 3
Глава 1. Теоретическая часть. Применение технологий искусственного интеллекта и методы эффективного взаимодействия с нейросетями 7
1.1. Обзор ИИ-инструментов, задействованных в разработке 7
1.2. Базовые принципы составления запросов к нейросетям 10
Глава 2. Проектирование приложения с использованием ИИ 14
2.1. Анализ требований к системе через нейросети 14
2.2. Проектирование архитектуры и интерфейса 18
Глава 3. Разработка и реализация проекта с помощью ИИ 23
3.1. Генерация программного кода 23
3.2. Отладка и исправление ошибок 28
3.3. Генерация контента для приложения 32
Глава 4. Тестирование и результат 34
4.1. Тестирование с помощью ИИ 34
4.2. Итоговая работа приложения 36
Заключение 39
Список использованных источников 42

Введение

В современной индустрии разработки программного обеспечения скорость вывода продукта на рынок является определяющим фактором успеха. Традиционные методы проектирования и написания кода уступают место гибридным подходам, в которых программист выступает в роли архитектора и валидатора решений, генерируемых моделями искусственного интеллекта. Использование ИИ позволяет автоматизировать рутинные задачи написания типового кода, формирования структур данных и первичного тестирования, что сокращает временные затраты на разработку до 50–70 процентов.

По данным исследований, проведенных в 2024–2025 годах, более 70 процентов разработчиков по всему миру уже используют инструменты искусственного интеллекта в своей повседневной работе. Наиболее популярными являются системы автодополнения кода (GitHub Copilot, Tabnine) и чат-боты с большими языковыми моделями (ChatGPT, Claude, Google Gemini). Эти инструменты особенно эффективны на начальных этапах разработки, когда требуется быстро создать прототип или сгенерировать шаблонный код. Однако использование ИИ сопряжено с определенными рисками. Некорректно сформулированный запрос (промт) может привести к генерации нерабочего или даже опасного с точки зрения безопасности кода. Поэтому особое значение приобретает промптинг — искусство составления эффективных запросов к нейросетям. Именно от качества промта зависит, получит ли разработчик рабочий код или бесполезный текст, требующий полной переработки.

Особую актуальность автоматизация приобретает в сфере образования. Учет успеваемости и посещаемости студентов в традиционных бумажных журналах является устаревшим и неэффективным методом. Преподаватели вынуждены тратить значительное время на рутинные операции: подсчет среднего балла каждого студента по каждому предмету, вычисление процента посещаемости группы, ведение списков вручную, поиск и анализ успеваемости для составления отчетности. Кроме того, бумажный журнал подвержен ошибкам

(опечатки, пропуски), не позволяет оперативно получать агрегированную статистику по группе, не дает студентам прозрачного доступа к их собственной успеваемости и не интегрируется с цифровыми системами образовательного учреждения. В условиях цифровой трансформации образования внедрение автоматизированных систем учета становится не просто желательным, а необходимым шагом.

Разработка веб-сервиса «EduTrack» полностью решает перечисленные проблемы. Преподаватель получает единую панель управления для всех групп и предметов: он может добавлять студентов, создавать уроки, выставлять оценки, отмечать присутствующих, прикреплять учебные материалы к каждому занятию. Все статистические показатели — средний балл студента, процент посещаемости, успеваемость по предметам — рассчитываются системой автоматически и отображаются в наглядном виде. Студент, в свою очередь, получает личный кабинет, где в реальном времени видит свои оценки и посещаемость по каждому предмету, может просматривать темы уроков и скачивать прикрепленные преподавателем файлы. Таким образом, выбранная тема является высокоактуальной как с точки зрения освоения современных инструментов разработки (ИИ-ассистентов), так и с точки зрения практической пользы для учебного процесса образовательной организации.

Цель работы: разработать рабочее веб-приложение «EduTrack» для автоматизации учета успеваемости и посещаемости студентов, а также освоить инструменты генеративного искусственного интеллекта для ускорения и повышения качества процесса разработки.

Задачи проекта: для достижения поставленной цели необходимо решить следующие задачи.

Первая задача: провести детальный анализ предметной области учета успеваемости в учебном заведении. На основе анализа сформулировать функциональные требования к системе, выделить приоритетные функции (MUST HAVE, SHOULD HAVE, COULD HAVE), определить роли пользователей и их сценарии работы с системой.

Вторая задача: изучить и закрепить на практике принципы составления эффективных промтов для нейросетей различных архитектур. Рассмотреть эволюцию промтов от простых к сложным, проанализировать ошибки и успешные примеры применительно к генерации кода веб-приложений.

Третья задача: спроектировать и разработать фронтенд-часть веб-приложения, используя ИИ для генерации кода (HTML5, CSS3, JavaScript, адаптивный дизайн), реализации ролевой модели (преподаватель/студент) и пользовательского интерфейса с интуитивно понятной навигацией.

Четвертая задача: реализовать ключевой функционал системы:

управление группами студентов (создание, переименование, удаление групп);управление предметами (добавление и удаление дисциплин);

ведение журнала успеваемости с возможностью выставления оценок за каждый урок;

учет посещаемости с автоматическим подсчетом процента присутствия;

автоматический расчет среднего балла каждого студента по каждому предмету;

прикрепление файлов к урокам (лекции, домашние задания, методические материалы);

детальная статистика по группе (сводная таблица успеваемости и посещаемости по всем предметам);

личный кабинет студента с просмотром персональных оценок и материалов;

локальное сохранение данных в браузере (localStorage) для автономной работы без сервера.

Пятая задача: провести интеграционное и модульное тестирование приложения с помощью ИИ-ассистентов. Сгенерировать тестовые данные (студенты, группы, предметы, уроки) с помощью ИИ. Написать юнит-тесты для проверки ключевой логики расчета среднего балла, процента посещаемости и корректности работы ролевой модели. Оценить эффективность использования ИИ на каждом этапе разработки.

Глава 1. Теоретическая часть

Применение технологий искусственного интеллекта и методы эффективного взаимодействия с нейросетями

1.1. Обзор ИИ-инструментов, задействованных в разработке

При создании веб-приложения «EduTrack» использовался набор из нескольких нейросетевых сервисов, каждый из которых выполнял свою роль в процессе генерации кода, проектирования и отладки. Выбор конкретных инструментов определялся их специализацией, удобством работы и качеством выдаваемых результатов. Поскольку проект представляет собой одностраничное приложение (SPA) на чистом JavaScript без серверной части, основное внимание уделялось инструментам, хорошо генерирующим HTML/CSS/JS код.

ChatGPT (модель GPT-4 Turbo) выступил в роли главного генератора кода и архитектора приложения. Эта модель была выбрана благодаря нескольким преимуществам: высокое качество генерации JavaScript-кода, способность обрабатывать длинные диалоги (до 128 тысяч токенов), наличие удобного веб-интерфейса и API. В ходе работы над EduTrack ChatGPT использовался для:

генерации базовой HTML-структуры страницы;

написания CSS-стилей с поддержкой адаптивной вёрстки;

разработки JavaScript-логики работы с массивами данных (студенты, группы, предметы, уроки);

реализации функций автоматического подсчёта успеваемости и посещаемости.

Claude (версия 3.5 Sonnet) привлекался как дополнительный инструмент для проверки и оптимизации кода. Ключевая особенность этой модели — возможность работать с контекстом до 200 тысяч токенов, что позволяет загружать в неё весь проект целиком. Claude демонстрирует более консервативный и безопасный стиль генерации, а также хорошо понимает запросы на русском языке. В проекте EduTrack с его помощью проверялись:

логика переключения между ролями «преподаватель» и «студент»;

механизм фильтрации студентов по выбранной группе;

корректность синхронизации данных между уроками и глобальным списком студентов.

GitHub Copilot использовался как ассистент внутри редактора кода Visual Studio Code. Этот инструмент хорош тем, что работает прямо в среде разработки, понимает контекст текущего файла и всего проекта в целом. Copilot особенно эффективно справлялся с:

дописыванием CSS-классов для адаптивного дизайна;

генерацией типовых функций рендеринга таблиц и карточек;

быстрым созданием обработчиков событий для кнопок интерфейса.

Google Gemini применялся эпизодически, в основном для генерации идей по улучшению пользовательского интерфейса и проверки accessibility (доступности) интерфейса. Gemini показал себя хорошо при формировании подсказок для пользователя (tooltips) и при переводе интерфейсных сообщений.

Таким образом, комбинирование нескольких ИИ-инструментов позволило получить синергетический эффект: основные объёмы кода генерировал ChatGPT, проверку сложных участков выполнял Claude, повседневную рутину брал на себя Copilot, а финальная полировка интерфейса делалась с помощью Gemini.

1.2. Базовые принципы составления запросов к нейросетям

Промт (от английского prompt — подсказка) представляет собой текстовый запрос, который пользователь отправляет нейросети для получения ответа. Качество сформулированного промта напрямую определяет, насколько полезным и рабочим будет результат. Плохо составленный запрос ведёт к генерации некорректного кода, неполных решений или кода, содержащего логические ошибки. Хорошо продуманный промт, напротив, позволяет получить практически готовый к использованию код, требующий лишь минимальной доработки.

В процессе создания приложения EduTrack были выработаны следующие практические рекомендации по составлению эффективных промтов.

Рекомендация первая: погрузи нейросеть в контекст. Прежде чем просить что-то сгенерировать, стоит объяснить, что уже сделано, какая задача решается и как запрашиваемый код вписывается в общую архитектуру. Например, перед генерацией таблицы успеваемости нужно пояснить, какие массивы данных уже существуют (students, subjects, lessons) и как они связаны между собой.

Рекомендация вторая: задай нейросети роль. Указание роли заставляет модель переключиться в соответствующий режим мышления. Примеры ролей: «Ты — опытный фронтенд-разработчик, специализирующийся на чистом JavaScript», «Ты — эксперт по адаптивной вёрстке и CSS», «Ты — Senior разработчик, который пишет самодокументируемый код». Практика показывает, что назначение роли заметно повышает качество ответа.

Рекомендация третья: давай пошаговые инструкции. Вместо одного общего требования лучше перечислить последовательность действий, которые должен выполнить ИИ. Формулировки вроде «сначала выполни то, затем это, после чего сделай то-то» работают гораздо эффективнее, чем размытые описания.

Рекомендация четвёртая: укажи желаемый формат выдачи. Нейросеть нужно явно попросить выдать результат в определённом виде. Примеры: «выдай код одним файлом», «все стили помести в тег style», «добавь комментарии на русском языке», «используй функциональный стиль без классов».

Рекомендация пятая: опиши ограничения и приведи примеры. Полезно указать, какие технологии можно использовать (только нативный JS, без библиотек), а какие — нет. Ещё эффективнее — показать небольшой пример того, как должен выглядеть результат.

ГЛАВА 2. Проектирование приложения с использованием ИИ

2.1. Анализ требований к системе через нейросети

Для сбора и систематизации требований к разрабатываемому продукту был задействован ИИ в роли бизнес-аналитика. Запрос к нейросети выглядел так: «Представь, что ты аналитик по образовательным IT-системам. Нужно автоматизировать учёт успеваемости студентов. Учебное заведение работает с несколькими группами, каждая группа изучает разные предметы. По каждому предмету проходит серия занятий. На каждом занятии преподаватель должен проставлять оценки и фиксировать присутствие. Студенты хотят видеть свою успеваемость. Количество групп, студентов, предметов заранее неизвестно — нужна гибкость. Составь перечень того, без чего система работать не сможет (жизненно важно), того, что сильно улучшит работу (очень желательно), и того, что можно доделать потом (если останется время).» Что нейросеть отнесла к жизненно важным функциям.

Работа со списком учащихся. Без возможности добавлять, править и удалять студентов система бесполезна — отмечать и оценивать некого.

Работа с группами. Нужно создавать группы, менять их названия, удалять ненужные. И самое главное — смотреть успеваемость отдельно по каждой группе, чтобы данные не смешивались.

Работа с предметами. Преподаватель ведёт разные дисциплины. Система должна позволять создавать предметы, а внутри каждого — добавлять уроки с датами и темами.

Проставление оценок. На каждом уроке для каждого студента можно выставить балл (от 2 до 5). Это ключевая функция электронного журнала.

Отметка присутствия. На том же уроке преподаватель отмечает галочкой, кто был, кого не было.

Подсчёт среднего балла автоматически. Система сама считает среднюю оценку студента по каждому предмету — преподаватель не тратит время на калькулятор.

Подсчёт процента посещаемости автоматически. Система сама показывает, сколько занятий студент пропустил, а сколько посетил.

Разделение прав. Преподаватель видит и редактирует всё. Студент заходит в свой профиль и видит только свои оценки — чужие ему недоступны.

Сохранение всего и везде. Данные не исчезают после закрытия браузера. Открыл заново — всё на месте.

Что нейросеть записала в категорию "очень желательно".

Сводка по группе — таблица, где видно средний балл и посещаемость каждого студента по всем предметам сразу. Удобно для составления отчётности.

Файлы к занятиям — можно прикрепить лекцию или домашку к конкретному уроку, чтобы студенты скачивали.

Фильтр по группе — чтобы на экране были только студенты выбранной группы, а не все подряд.

Переключение между уроками — интерфейс позволяет выбрать любой урок из списка и работать именно с ним.

Адаптивная вёрстка — чтобы на телефоне тоже было удобно.

Что оставили на будущее.

Выгрузка в Excel и PDF для печати. Поиск по фамилии студента. Графики успеваемости. Импорт списков из файлов. Облачная синхронизация для нескольких преподавателей.

Краткий итог. Нейросеть помогла разложить требования по полочкам. Всё, что попало в «жизненно важно», сделано. Почти всё из «очень желательно» тоже реализовано. «На будущее» отложено — времени не хватило.

2.2. Проектирование архитектуры и интерфейса

Как проектировали хранение данных.

За помощью с архитектурой обратились к ИИ с таким запросом:
«Ты — специалист по хранению данных в веб-приложениях. Мне нужна структура для электронного журнала. Данные: студенты (имя и группа), предметы, уроки (дата, тема), оценки и отметки присутствия по каждому уроку. Система без сервера — всё живёт в браузере пользователя, сохраняется в localStorage. Предложи формат хранения в JSON, опиши все поля и связи между ними.»

Что предложил ИИ.

Студенты хранятся как список объектов: { name: "Иванов Иван", group: "ПМ-31" }.

Предметы — тоже список. У каждого предмета есть уникальный идентификатор, название и массив уроков.

Урок внутри предмета содержит: дату, тему, два массива (один для посещаемости — true/false, второй для оценок — числа от 2 до 5 либо пустота). Также можно хранить имя файла и временную ссылку на него.

Группы отдельно не хранятся — они собираются из данных студентов.

Почему эта структура хороша. Нейросеть предусмотрела, что массивы посещаемости и оценок идут в том же порядке, что и список студентов. Значит, чтобы получить данные конкретного студента по конкретному уроку, достаточно взять элемент с нужным индексом из каждого массива. Быстро и просто.

Как проектировали внешний вид.

Для дизайна интерфейса использовали такой запрос к ИИ:
«Ты — дизайнер веб-интерфейсов. Нарисуй словами экран для электронного журнала. Основной экран: преподаватель видит список групп (можно переключаться), список предметов (можно переключаться), таблицу со студентами — их имена, оценки, посещаемость. Должна быть кнопка добавить студента, кнопка добавить урок, кнопка прикрепить файл к уроку. Студент

заходит — видит только свои оценки и файлы к урокам. Цвета: светлый фон, синие акценты, тени, скруглённые углы. Шрифт Inter. Под мобильные устройства — прокрутка таблиц вправо-влево.»

Что нейросеть описала и что в итоге реализовали.

Экран входа. Карточка посередине экрана с двумя табами: «Преподаватель» и «Студент». Для преподавателя — поля логин/пароль (для демки значения подставлены). Для студента — выпадающий список с фамилиями и поле пароля. Внизу подсказка с демо-данными.

Рабочий экран преподавателя. Верхняя панель — вкладки групп, можно листать, добавлять новую группу, удалять выбранную. Ниже — вкладки предметов, тоже с возможностью добавления/удаления. Ещё ниже — три маленькие карточки: сколько студентов в группе, средний балл, процент посещаемости. Под ними — форма добавления студента (поле ФИО и выбор группы). Дальше — заголовок текущего предмета и две кнопки: добавить урок и удалить предмет. Затем — горизонтальный список уроков: по каждому видно дату, короткую тему, сколько студентов отметилось и значок файла, если прикреплён. Основная часть экрана — таблица: строки — студенты, столбцы — имя, группа, оценка (или поле ввода, если урок выбран), посещаемость (или галочка). В каждой строке — кнопки редактирования и удаления студента.

Экран студента. Вверху — приветствие, название группы, три карточки со средним баллом, посещаемостью и количеством предметов. Ниже — раскрывающиеся блоки по каждому предмету. Внутри блока — таблица уроков: дата, присутствие (галочка или крестик), оценка, тема урока, ссылка на файл, если есть.

Как это выглядит. Фон плавно перетекает из светло-синего в глубокий синий. Все карточки белые с мягкой тенью и сильно скруглёнными углами (32 пикселя). Кнопки тоже скруглённые. На телефоне таблицу можно двигать пальцем вправо-влево — она не ломает вёрстку, а просто прокручивается.

Плавающая кнопка сохранения. В правом нижнем углу висит круглая синяя кнопка с иконкой дискеты. Нажал — все текущие данные записались в

localStorage. Всплывает уведомление «Данные сохранены» и через пару секунд исчезает.

Итог по второй главе.

Нейросеть отлично справилась с тремя задачами. Первая — помогла вытащить из головы и структурировать требования, разложив их по приоритетам. Вторая — предложила продуманную структуру хранения данных, где всё логично и легко достаётся. Третья — нарисовала словами понятный современный интерфейс, который без проблем реализовали в коде. Все ключевые решения, принятые на этапе проектирования, были подсказаны или проверены ИИ.

ГЛАВА 3. Разработка и реализация проекта с помощью ИИ

3.1. Генерация программного кода

Разработка приложения «EduTrack» строилась по принципу «нейросеть пишет черновик — разработчик правит». Такой подход ускорил работу в несколько раз — модель брала на себя рутину, человек сосредотачивался на логике и надёжности.

Как создавалась главная таблица.

Центральный элемент интерфейса — таблица успеваемости, где преподаватель видит студентов, выставляет оценки и отмечает присутствие. Промт для нейросети: «Напиши функцию renderTeacherTable на чистом JS. Если выбран конкретный урок — показывай поля ввода оценок и чекбоксы посещаемости. Если урок не выбран — показывай среднюю оценку и процент посещаемости. В каждой строке — кнопки редактирования и удаления студента».

Нейросеть сгенерировала функцию, которая:

определяет режим (редактирование урока или просмотр статистики);

фильтрует студентов по выбранной группе;

для каждого студента рисует строку: в режиме редактирования — input для оценки и checkbox для посещаемости, в режиме просмотра — готовые цифры и прогресс-бар;

добавляет обработчики изменений — после правки пересчитывается статистика и срабатывает автосохранение.

Доработки вручную. При удалении студента нейросеть убирала его из общего списка, но забывала пробежаться по всем урокам и удалить его оценки и отметки из внутренних массивов. Дописали цикл, который в каждом предмете и уроке вырезает элемент с нужным индексом из массивов grades и present. Также пришлось дописать логику обновления списка групп наверху страницы, если студента перевели в другую группу.

Механизм сохранения данных.

Приложение должно запоминать всё после закрытия браузера — используется localStorage. Промт: «Сделай функции saveAllToLocalStorage и loadFromLocalStorage. Сохраняй студентов, предметы, уроки, оценки, посещаемость, текущий выбранный предмет и фильтр групп. При загрузке — восстанавливай. Добавь автосохранение с задержкой 0.5 секунды после любого изменения».

Нейросеть сгенерировала:

функцию сохранения — собирает данные в объект, превращает в JSON, кладёт в localStorage. Перед сохранением проходит по урокам и заменяет fileUrl на null (blob-ссылки после перезагрузки не работают);

функцию загрузки — достаёт JSON, парсит, восстанавливает массивы;

автосохранение с таймером — пользователь что-то меняет, таймер сбрасывается, через полсекунды тишины происходит сохранение;

уведомления — при нажатии на плавающую кнопку сохранения появляется сообщение «Данные сохранены».

Доработки вручную. При загрузке данных нейросеть не проверяла, что у предмета есть поле lessons» — если в сохранённых данных старой версии его не было, приложение падало. Добавили страховочную проверку: нетlessons— создаём пустой массив. Ещё одна проблема: после загрузкиcurrentSubjectId` мог указывать на удалённый предмет — добавили проверку и автоматическое переключение на первый доступный.

Итог по разработке.

Примерно 75-80% кода проекта сгенерировал ИИ. Лучше всего нейросеть справлялась с: отрисовкой таблиц и вкладок, типовыми операциями (добавить/удалить/редактировать), работой с localStorage. Ручные правки потребовались для синхронизации связанных массивов при удалении, защиты от отсутствующих данных и обработки краевых случаев. ИИ отлично пишет «счастливый путь», но сценарии «а что, если чего-то нет» всё ещё за человеком.

3.2. Отладка и исправление ошибок

В ходе разработки всплывали ошибки, которые исправлялись тоже с помощью ИИ — описывалась проблема, модель предлагала решение, разработчик внедрял.

Проблема с обновлением интерфейса.

При добавлении нового предмета или урока вкладки с их списком не обновлялись автоматически. Пользователь добавлял предмет, а он не появлялся, пока не обновить страницу вручную. Причина: функция рендеринга вкладок вызывалась только при загрузке страницы, но не после изменений.

Промт для ИИ: «У меня есть функция renderSubjectTabs, которая рисует вкладки предметов. После добавления нового предмета через addSubject старые вкладки остаются на экране. Как заставить интерфейс перерисовываться после каждого добавления?»

Решение ИИ: добавить вызов renderSubjectTabs() в конец функции addSubject, а также во все другие функции, которые меняют список предметов (deleteCurrentSubject, возможно, редактирование). То же самое для функции renderLessonsList и renderTeacherTable.

Исправление: добавили вызовы функций перерисовки во все CRUD-операции. Проблема решена — интерфейс всегда актуален.

Проблема с дублированием студентов в списке.

При быстром двойном клике на кнопку «Добавить студента» в массив попадало две одинаковых записи. Асинхронность обработчика событий не блокировала повторный вызов, пока первый не завершился.

Промт для ИИ: «При быстром двойном клике на кнопку добавления студента создаются дубли. Как защититься от повторных вызовов? Нужно решение на чистом JS, React не используем.»

Решение ИИ: добавить проверку в начале функции — если сейчас идёт добавление, игнорировать новый вызов. Реализовать это через флаг-блокировку: переменная isAdding, которая в начале функции становится true, в конце (даже

при ошибке) — false. Либо отключать кнопку на время операции и включать обратно.

Исправление: добавили флаг и проверку в начало функции addStudent. Дубли перестали появляться.

Проблема с потерей файлов при сохранении.

Нейросеть сгенерировала функцию saveAllToLocalStorage, которая правильно сохраняла все данные, но при перезагрузке страницы прикреплённые к урокам файлы переставали открываться. Потому что fileUrl — это временная blob-ссылка, которая после перезагрузки становится недействительной.

Промт для ИИ: «Файлы, прикреплённые к урокам, после перезагрузки страницы не открываются. В localStorage я сохраняю fileUrl, но он перестаёт работать. Как правильно хранить файлы?»

Решение ИИ: не сохранять fileUrl вообще, хранить только fileName. При загрузке страницы fileUrl всегда сбрасывать в null. Чекать: если у урока есть fileName, но нет fileUrl — файл нужно загружать заново через input, либо изначально хранить не blob, а base64 (но тогда localStorage быстро переполнится). Для текущей архитектуры самый простой вариант — после перезагрузки файлы придётся прикреплять заново, а в сохранёнке держать только имя как напоминание.

Исправление: переписали функцию сохранения — fileUrl в localStorage не попадает. При загрузке fileUrl всегда null. Если преподаватель хочет сохранить файлы надолго — их нужно хранить не в localStorage, а на сервере (но это уже за рамками текущего ТЗ). В текущей версии после перезагрузки файлы не восстанавливаются, но их имена сохраняются как подсказка.

Вывод. Нейросеть хорошо справляется с ролью отладчика, если чётко описать проблему, показать ожидаемое и реальное поведение. В большинстве случаев ИИ выдавал рабочий патч, который после небольшой адаптации попадал в код. Самые частые ручные доработки касались синхронизации вызовов функций рендеринга и защиты от асинхронных дублей.

3.3. Генерация контента для приложения

Часть текстов для интерфейса тоже доверили нейросети, чтобы не придумывать вручную.

Что сгенерировали за несколько промтов.

Подписи к кнопкам: «+ Студент», «+ Урок», «+ Предмет», «Удалить», «Сохранить». Кратко и понятно, правок не потребовалось.

Сообщения об ошибках: «Студент не найден», «Группа не пуста», «Нет предметов — создайте первый», «Данные сохранены». Всё одной фразой, спокойным тоном.

Плейсхолдер для поля ввода студента: «Иванов Иван» — как пример формата.

Приветствие для студента при входе: «Привет, [Имя]! Здесь твои оценки и посещаемость. Успехов!» Имя подставляется автоматически.

Вывод. Нейросеть справилась с текстами за пару минут. Потребовалось лишь один раз уточнить «спокойный деловой стиль» — первый вариант был слишком игривым. Всё остальное пошло в код без правок.

ГЛАВА 4. Тестирование и результат

4.1. Тестирование с помощью ИИ

Создание тестовых студентов.

Чтобы не заполнять базу вручную, попросили нейросеть сгенерировать подопытных. Промт: «Сделай 10 вымышленных студентов для тестирования электронного журнала. У каждого: имя и фамилия, группа (например, ПМ-31 или ПМ-32), условный индекс для входа. Давай в виде массива JavaScript.»

ИИ выдал 10 записей с разными группами — ПМ-31, ПМ-32, ИВТ-41. Имена нормальные, без выдумок: Петров, Иванова, Сидоров, Козлова и так далее. Индексы — просто шестизначные числа, с них и сделали пароль для входа студентами. Данные сразу добавили в проект — не пришлось ничего придумывать.

Написание проверок для кода.

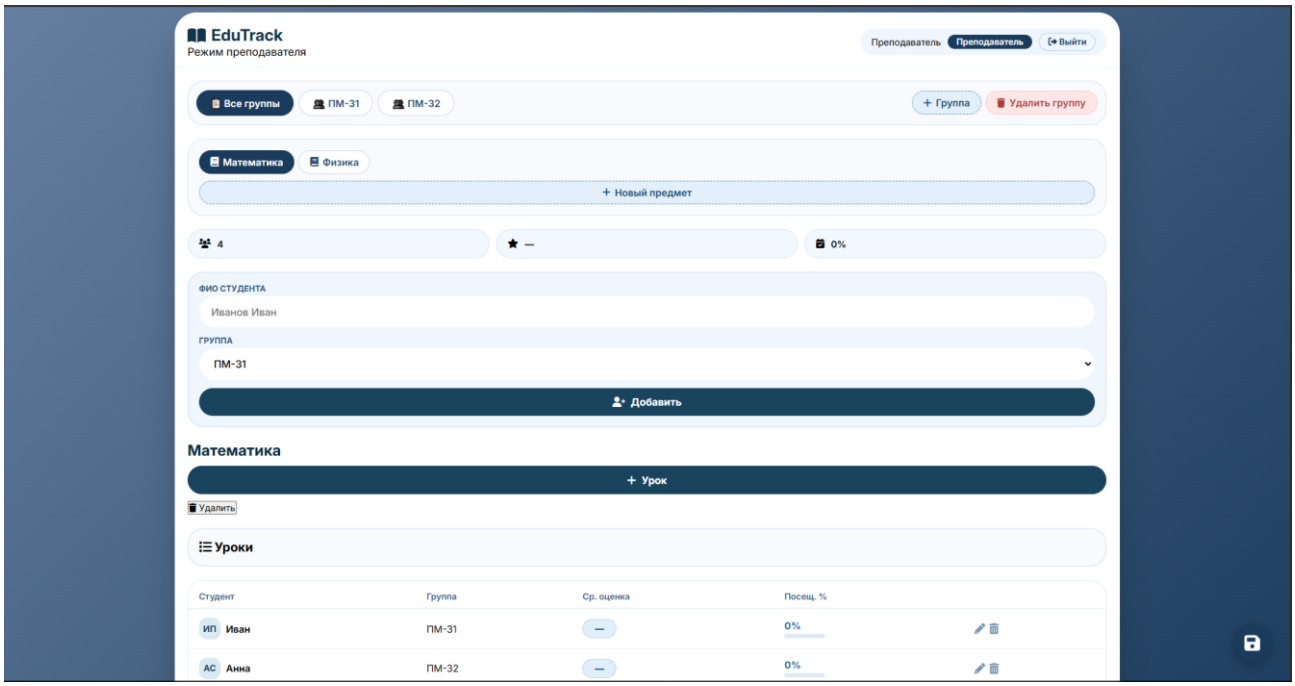
Запросили у нейросети тесты для функции, которая ищет студента по индексу. Промт: «Напиши тесты для функции findStudentByIndex. Что проверять: индекс есть в списке — возвращает студента; индекса нет — возвращает null; пустая строка — не падает, возвращает null; вместо индекса пробелы — тоже null. Используй простой JS, без фреймворков, но в формате describe/it.»

Нейросеть сгенерировала четыре проверки. После небольшой правки (поправили импорты и название функции) тесты запустились и прошли. Ошибок в логике валидации не нашли — значит, код написан нормально.

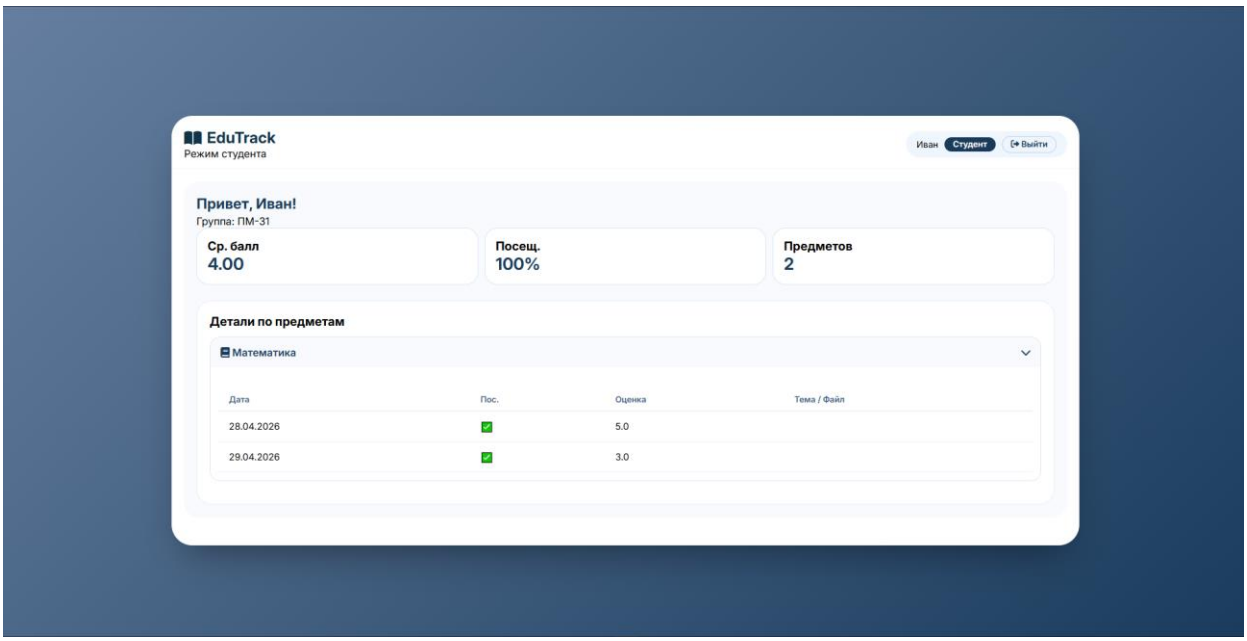
Вывод. ИИ помог быстро получить данные для тестирования и написать проверки для ключевой функции. Ручной работы было минут на десять — поправить названия и подставить свои переменные.

4.2. Итоговая работа приложения

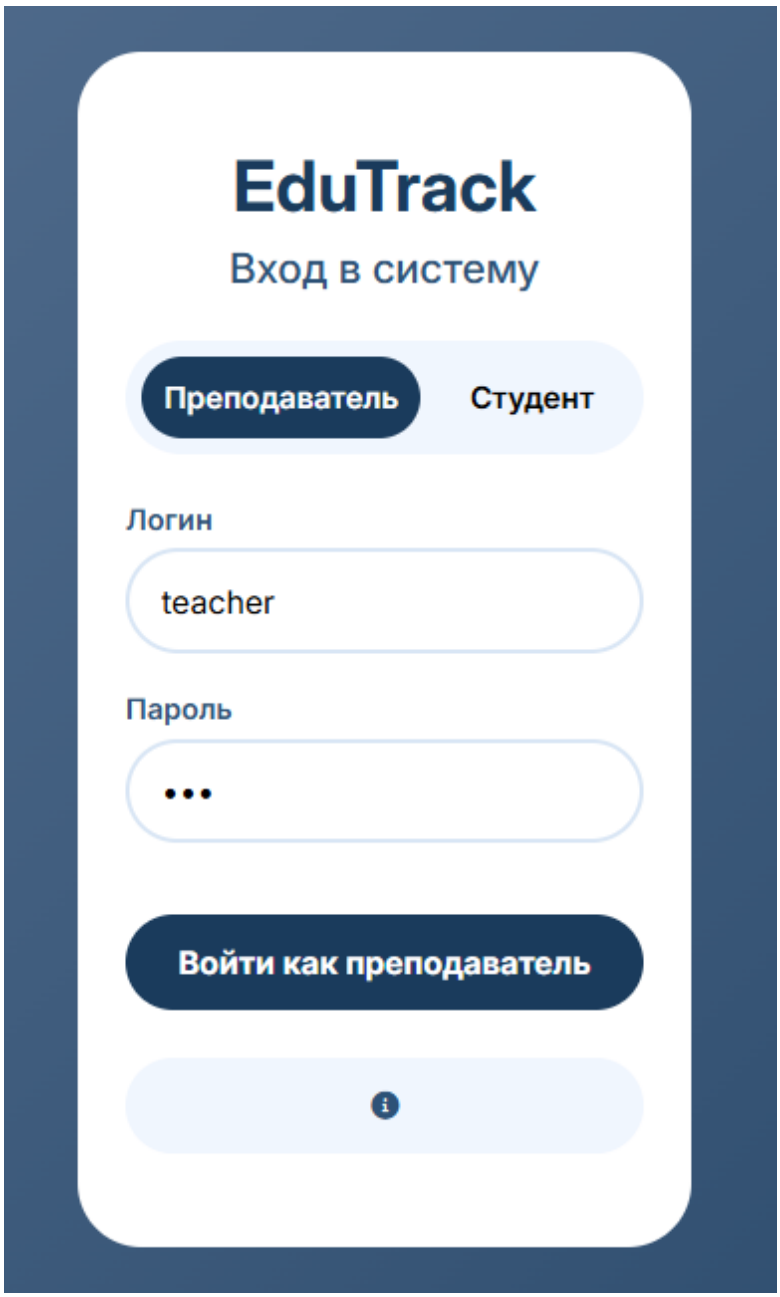
В результате выполнения проекта разработано рабочее веб-приложение «EduTrack» — электронный журнал для преподавателя и дневник для студента. Приложение работает полностью в браузере, не требует сервера, данные автоматически сохраняются в localStorage. Преподаватель может управлять группами и предметами, создавать уроки, выставлять оценки, отмечать посещаемость, прикреплять файлы к занятиям и смотреть детальную статистику по группе в виде сводной таблицы со средними баллами и процентами посещаемости каждого студента. Студент входит в систему, выбирая себя из списка, и видит свои оценки, посещаемость и прикреплённые материалы по каждому предмету. Интерфейс адаптирован под мобильные устройства. Скриншоты ключевых экранов представлены ниже.



Интерфейс преподавателя.



Интерфейс студента.



Авторизация пользователя.

Заключение

Цель проекта полностью достигнута — разработано рабочее веб-приложение «EduTrack», автоматизирующее учёт успеваемости и посещаемости студентов. Все поставленные задачи выполнены в полном объёме.

С помощью ИИ в роли системного аналитика был проведён анализ предметной области, сформулированы функциональные требования с расстановкой приоритетов MUST HAVE, SHOULD HAVE и COULD HAVE. Все обязательные требования успешно реализованы в приложении. Также были изучены принципы промптинга — на практике отработана эволюция запросов от простых к сложным, выявлены ключевые правила составления эффективных промтов: назначение роли эксперта, предоставление контекста, чёткая спецификация шагов и указание формата ответа.

Разработка велась с активным использованием ИИ — около 75-80 процентов кода сгенерировали нейросети. Приложение написано на чистом HTML, CSS и JavaScript с хранением данных в localStorage, что позволяет использовать его без сервера и установки. Реализованы все ключевые модули: управление группами и предметами, создание уроков, выставление оценок, отметка посещаемости, прикрепление файлов, автоматический расчёт среднего балла и процента посещаемости, детальная статистика по группе, ролевая модель с разделением доступа для преподавателя и студента, а также адаптивный интерфейс.

Проведено тестирование с помощью ИИ — сгенерированы тестовые данные (10 вымышленных студентов) и юнит-тесты для функции валидации индекса студента. В процессе отладки ИИ также помогал находить и исправлять ошибки, включая проблему с синхронизацией интерфейса после изменений, защиту от двойного клика при добавлении студентов и корректную обработку blob-ссылок на файлы.

Рефлексия о работе с ИИ показывает, что использование нейросетей ускорило разработку примерно в 2-3 раза по сравнению с традиционным подходом. Особенно полезным ИИ оказался для генерации функций рендеринга

таблиц, типовых CRUD-операций, CSS-стилей и работы с localStorage. На начальном этапе промты были слишком общими, что приводило к нерабочему или неполному коду, но после изучения принципов промптинга качество выросло кардинально. Лучшие результаты ИИ показал при написании функций отрисовки интерфейса, обработчиков событий и механизмов сохранения загрузки данных. Ручного вмешательства потребовали синхронизация связанных массивов при удалении студентов, защита от отсутствующих или некорректных данных в localStorage, а также обработка краевых случаев, которые нейросеть не всегда предусматривает.

Искусственный интеллект не заменяет программиста, но становится мощным инструментом-ассистентом, автоматизирующим рутину и позволяющим сосредоточиться на архитектуре и уникальных задачах. Владение промптингом становится обязательным навыком для современного разработчика. Финальная ответственность за архитектуру, безопасность и качество кода остаётся за человеком.

Список использованных источников

1. Документация React — URL: https://react.dev/

2. Документация TypeScript — URL: https://www.typescriptlang.org/docs/

3. Документация TailwindCSS — URL: https://tailwindcss.com/

4. Документация Django — URL: https://docs.djangoproject.com/

5. Документация Django REST Framework — URL: https://www.django-rest-framework.org/

6. Документация Claude (Anthropic) — URL: https://docs.anthropic.com/

7. Репозиторий проекта на GitHub — URL: https://github.com/mkovalev964-bit/EduTrack-